

OpenClaw 个人智能体网关 — 完整运维手册

机器: WLY\10979 (Windows 11) | **OpenClaw 版本:** v2026.6.6

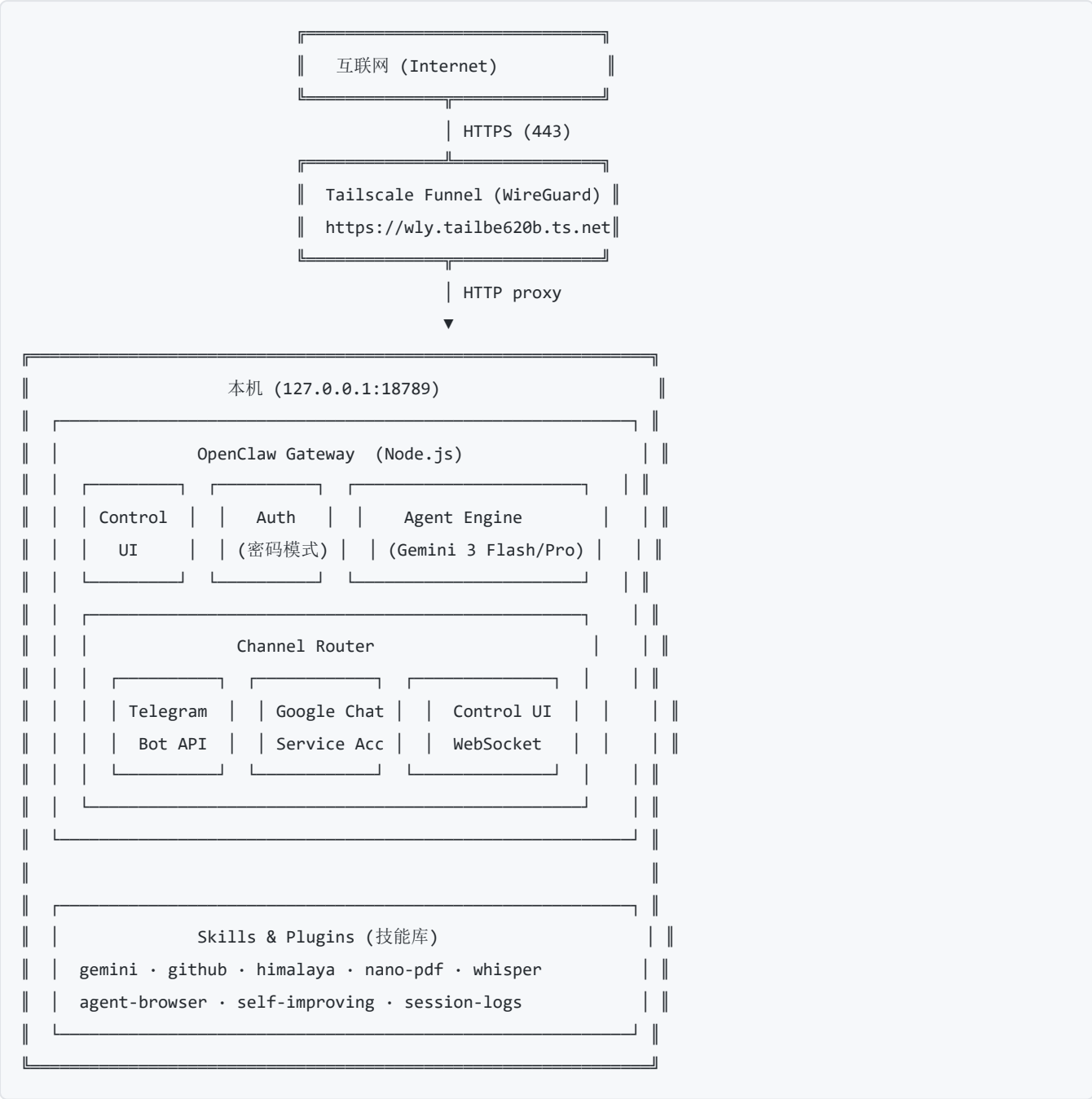
网关端口: 18789 (loopback) | 公网入口: <https://wly.tailbe620b.ts.net>

最后审计: 2026-06-16 | 审计人: Antigravity Agent (Google DeepMind)

目录

- 系统架构总览
- 核心组件清单
- 静默自启方案 (已修复)
- 安全认证与免登录运行
- Tailscale Funnel 公网隧道
- Telegram 渠道配置
- Google Chat 渠道配置
- 技能与插件生态
- 缺陷审计与已修复项
- 调优建议
- 灾难恢复与重装指南
- 运维速查命令
- 文件清单
- 变更历史

1. 系统架构总览



核心运行链路 (开机 → 响应消息, 全程零人工干预) :

```
Windows Boot → Task Scheduler "OpenClaw Gateway" (BootTrigger +30s)
→ wscript.exe (隐藏窗口)
→ openclaw_run_hidden.vbs
→ gateway.cmd
→ node.exe openclaw gateway --port 18789
→ Tailscale Funnel 反代注入 (HTTPS)
→ Telegram Bot / Google Chat 渠道就绪
```

2. 核心组件清单

组件	版本/路径	用途
OpenClaw	v2026.6.6	个人智能体网关框架
Node.js	C:\Program Files\nodejs\node.exe	OpenClaw 运行时
OpenClaw 包	C:\Users\10979\AppData\Roaming\npm\node_modules\openclaw\dist\index.js	npm 全局安装
配置文件	C:\Users\10979\.openclaw\openclaw.json	网关核心配置
启动脚本	C:\Users\10979\.openclaw\gateway.cmd	启动入口 (OpenClaw 自动生成)
工作空间	C:\Users\10979\.openclaw\workspace	技能/记忆/工作区
Tailscale	C:\Program Files\Tailscale\ (服务模式, Automatic)	WireGuard 隧道 + Funnel 公网入口
计划任务	OpenClaw Gateway	静默自启调度
VBS 包装器	E:\RamdiskGuardian\openclaw_run_hidden.vbs	零窗口启动器
任务 XML	E:\RamdiskGuardian\openclaw_task.xml	任务定义 (可复现部署)

3. 静默自启方案（已修复）

3.1 方案原理

OpenClaw 的 Windows 自启需要同时解决三个维度的问题：

维度	问题	解法
何时启动	必须在系统启动时运行，不依赖用户登录	BootTrigger (非 LogonTrigger)
以谁身份	需要用户身份访问 ~/.openclaw/ 配置，但不能有交互窗口	S4U LogonType (非 InteractiveToken)
窗口隐藏	CMD/PowerShell 黑框不能闪现	VBScript WScript.Shell.Run windowStyle=0

3.2 任务定义详情

```
<Task version="1.2">
  <Principals>
    <Principal id="Author">
      <UserId>S-1-5-21-1536040469-1817842872-232824913-1001</UserId>
      <LogonType>S4U</LogonType>                                <!-- 无交互/无窗口 -->
      <RunLevel>HighestAvailable</RunLevel>                    <!-- 管理员权限 -->
    </Principal>
  </Principals>
  <Triggers>
    <BootTrigger>
      <Delay>PT30S</Delay>                                     <!-- 等网络就绪 -->
    </BootTrigger>
  </Triggers>
  <Settings>
    <Hidden>true</Hidden>                                       <!-- 任务管理器中隐藏 -->
    <ExecutionTimeLimit>PT0S</ExecutionTimeLimit>             <!-- 永不超时 -->
    <RestartOnFailure>
      <Count>3</Count>                                         <!-- 崩溃自动重试3次 -->
      <Interval>PT1M</Interval>                                <!-- 每次间隔60秒 -->
    </RestartOnFailure>
    <StartWhenAvailable>true</StartWhenAvailable>             <!-- 错过的启动会补执行 -->
    <MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
  </Settings>
  <Actions Context="Author">
    <Exec>
      <Command>wscript.exe</Command>                           <!-- 隐藏窗口包装器 -->
      <Arguments>"E:\RamdiskGuardian\openclaw_run_hidden.vbs"</Arguments>
      <WorkingDirectory>C:\Users\10979\.openclaw</WorkingDirectory>
    </Exec>
  </Actions>
</Task>
```

3.3 S4U LogonType 说明

S4U (Service-for-User) 是 Windows 计划任务的一种特殊登录方式： - **不需要存储密码**：使用 Kerberos S4U 协议，不像 **Password** 登录类型那样需要在计划任务中保存明文密码 - **不需要交互会话**：不在桌面上创建窗口站 (Window Station)，因此不会弹出任何窗口 - **会加载用户配置文件**：**%USERPROFILE%**、**%APPDATA%** 等路径正常可用，User 级别的环境变量可以读取 - **无网络凭据限制**：S4U 令牌无法访问远程网络资源（如 SMB 共享），但 OpenClaw 只需要本地 loopback + Tailscale，不受影响

3.4 30 秒延迟的必要性

启动延迟 **PT30S** 的原因： 1. **Tailscale 服务需要时间建立 WireGuard 隧道**：**Tailscale** 服务虽然是 **Automatic** 启动，但实际连接建立需要约 10-20 秒 2. **DNS 解析需要网络就绪**：Telegram Bot API 需要解析 **api.telegram.org** 3. **Node.js 冷启动**：首次启动 Node.js 加载 OpenClaw 模块约需 5-10 秒

4. 安全认证与免登录运行

4.1 认证模式

```
{
  "gateway": {
    "auth": { "mode": "password" },
    "controlUi": { "allowInsecureAuth": true }
  }
}
```

- **mode: "password"** : 使用环境变量 `OPENCLAW_GATEWAY_PASSWORD` 中的密码进行认证
- **不是 browser 模式**: 不需要在浏览器中手动完成 OAuth/SSO 登录, 实现完全 headless 运行
- **allowInsecureAuth: true** : 允许通过 HTTP (非 HTTPS) 传输密码。这在 loopback 场景下是安全的, 因为 Tailscale Funnel 在公网侧已提供 TLS 加密

4.2 环境变量

变量名	级别	值	状态
OPENCLAW_GATEWAY_PASSWORD	~~User~~ → Machine	wlySecureClaw2026!	✅ 已提升

重要修复: 原始安装将密码设在 User 级别环境变量。在 `S4U + BootTrigger` 场景下, 虽然 S4U 会加载用户配置文件, 但为了消除启动早期的竞争条件 (用户配置文件尚未加载完毕时 Node.js 已开始读取环境变量), 已将其**提升至 Machine 级别**。Machine 级别的环境变量 由系统注册表直接读取, 在系统引导阶段即可用。

4.3 Telegram 免登录验证流程

```
开机 → Gateway 启动 → 读取 OPENCLAW_GATEWAY_PASSWORD (Machine 级别)
    → 使用 password auth 初始化 → 启动 Telegram Bot Polling
    → 收到消息 → 检查 allowFrom: [8320970051] → 响应
```

无需人工操作: 整个流程不需要在浏览器中打开 Control UI 做任何登录操作。

5. Tailscale Funnel 公网隧道

5.1 当前配置

```
Tailscale 状态:
  主机名:  wly
  IP:      100.115.124.21
  Funnel:  ON
  域名:    https://wly.tailbe620b.ts.net

路由:
  https://wly.tailbe620b.ts.net → http://127.0.0.1:18789 (Funnel ON)
```

5.2 Funnel 与 Gateway 的配合

```
{
  "gateway": {
    "mode": "local",
    "port": 18789,
    "bind": "loopback",          // 仅监听 127.0.0.1，外部无法直连
    "tailscale": {
      "mode": "funnel",          // 自动启用 Tailscale Funnel
      "resetOnExit": false       // 退出时不关闭 Funnel
    }
  }
}
```

- `bind: "loopback"` : Gateway 只在 `127.0.0.1:18789` 监听，防火墙无需开放任何端口
- `tailscale.mode: "funnel"` : Gateway 启动时自动注册 Tailscale Funnel 路由
- `resetOnExit: false` : 即使 Gateway 临时重启，Funnel 路由也保持活跃，避免 DNS 更新延迟

5.3 安全模型

互联网 → Tailscale Funnel (TLS + WireGuard) → 127.0.0.1:18789

公网暴露面: 仅 Tailscale 的 DERP relay (无直接 IP 暴露)

认证层: OPENCLAW_GATEWAY_PASSWORD

授权层: Telegram allowFrom / Google Chat serviceAccount

6. Telegram 渠道配置

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "botToken": "8857353244:AAHswW0qeNXsUJAFBAXOT7-L00M6ekrUQiI",
      "dmPolicy": "open",
      "allowFrom": [8320970051]
    }
  }
}
```

参数	值	说明
botToken	8857353244:AAH...QiI	Telegram Bot Father 颁发的 API Token
dmPolicy	open	允许 DM 私聊
allowFrom	[8320970051]	仅允许此 Telegram User ID 发送消息

Owner 命令权限

```
{
  "commands": {
    "ownerAllowFrom": ["telegram:8320970051"]
  }
}
```

仅 Telegram 用户 `8320970051` 拥有 Owner 级别的命令执行权限。

7. Google Chat 渠道配置

```
{
  "channels": {
    "googlechat": {
      "enabled": true,
      "serviceAccount": "{",
      "audienceType": "app-url",
      "audience": "https://wly.tailbe620b.ts.net/googlechat"
    }
  }
}
```

⚠️ 注意: `serviceAccount` 字段当前值为 `"{"`，这看起来像一个不完整的 JSON 服务账号配置。如果 Google Chat 渠道需要正常工作，请用完整的 Google Cloud Service Account JSON 替换此值。当前 Telegram 渠道已独立运行，Google Chat 渠道的配置不影响 Telegram。

8. 技能与插件生态

8.1 已安装技能

技能	用途
gemini	Google Gemini 大模型直接调用
github	GitHub 代码仓库操作 (PR/Issue/搜索)
himalaya	邮件收发 (IMAP/SMTP)
nano-pdf	PDF 文档解析与生成
openai-whisper	语音转文字 (Whisper API)
agent-browser	浏览器自动化 (Clawdbot)
self-improving	自我学习与优化 (记忆/反思/心跳)
session-logs	会话日志记录

8.2 已启用插件

插件	状态
google	✔ 启用
googlechat	✔ 启用
parallel	✔ 启用

8.3 模型配置

```
{
  "agents": {
    "defaults": {
      "model": { "primary": "google/gemini-3-flash-preview" },
      "models": {
        "google/gemini-3.1-pro-preview": {},
        "google/gemini-3-flash-preview": {}
      }
    }
  },
  "auth": {
    "profiles": {
      "google:default": { "provider": "google", "mode": "api_key" }
    }
  }
}
```

- **主模型:** google/gemini-3-flash-preview (快速响应)
- **备选模型:** google/gemini-3.1-pro-preview (深度推理)
- **认证方式:** Google API Key

8.4 安全策略 — 禁用命令

```
{
  "gateway": {
    "nodes": {
      "denyCommands": [
        "camera.snap", "camera.clip", "screen.record",
        "contacts.add", "calendar.add", "reminders.add",
        "sms.send", "sms.search"
      ]
    }
  }
}
```

已禁止摄像头/录屏/联系人/短信等敏感操作，防止智能体越权。

9. 缺陷审计与已修复项

9.1 已修复的严重缺陷

#	缺陷描述	原值	修复值	严重度
1	触发器使用 <code>LogonTrigger</code> ，需要用户登录才启动	<code>LogonTrigger</code>	<code>BootTrigger +30s</code>	致命
2	<code>InteractiveToken</code> 登录类型会弹出黑框窗口	<code>InteractiveToken</code>	<code>S4U</code>	致命
3	未设置最高权限运行级别	默认(Limited)	<code>HighestAvailable</code>	严重
4	<code>OPENCLAW_GATEWAY_PASSWORD</code> 仅在 User 级别	User 环境变量	Machine 环境变量	中等
5	无失败重启机制	无	3次/60秒	中等
6	无启动延迟，可能在网络未就绪时启动	无延迟	<code>PT30S</code>	中等
7	任务设置缺少 <code>Hidden</code> 标记	无	<code>Hidden=true</code>	低
8	无 <code>StartWhenAvailable</code> ，错过的启动不会补执行	无	启用	低

9.2 当前残留风险

#	风险	严重度	缓解措施
1	Google Chat <code>serviceAccount</code> 字段不完整 ("{ ")	中等	不影响 Telegram，按需修复
2	<code>gateway.cmd</code> 中 <code>node.exe</code> 路径硬编码，升级 Node.js 后可能断裂	低	Node.js 默认安装路径稳定
3	OpenClaw 全局 npm 安装路径依赖 <code>%APPDATA%</code> ，S4U 下理论可访问	低	<code>gateway.cmd</code> 使用绝对路径
4	VBS 启动器存放在 <code>E:\RamdiskGuardian</code> ，E: 盘不可用时任务失败	低	E: 是 SSD 固定分区
5	Tailscale 证书续期需 Tailscale 服务在线	低	Tailscale 服务已设为 Automatic

10. 调优建议

10.1 性能调优

✅ 已实施

- `bind: "loopback"`：不暴露公网端口，减少 TCP 连接开销
- `tailscale.resetOnExit: false`：避免 Funnel 路由频繁注册/注销
- BootTrigger 30 秒延迟**：平衡启动速度与网络就绪

💡 建议实施

项目	建议	优先级	说明
Node.js 堆内存	在 <code>gateway.cmd</code> 中添加 <code>--max-old-space-size=512</code>	中	当前 node.exe 内存约 400MB，限制堆可防止内存泄漏膨胀
日志轮转	添加 <code>OPENCLAW_LOG_LEVEL=warn</code> 环境变量	低	减少生产环境的日志输出量
Gemini 模型选择	日常使用 Flash，复杂任务切 Pro	—	已按此配置，无需更改
会话隔离	<code>dmScope: "per-channel-peer"</code>	—	已按最佳实践配置

10.2 安全加固建议

项目	当前状态	建议	说明
密码强度	18字符混合	✓ 足够	wlySecureClaw2026! 长度和复杂度满足要求
Bot Token 保护	明文存储在 openclaw.json	建议备份	若泄露需通过 @BotFather /revoke 重置
allowFrom 限制	仅允许单个 User ID	✓ 最小权限	仅 owner 可以发消息
denyCommands	已禁止摄像头/短信等	✓ 良好	覆盖主要敏感操作
HTTPS	Tailscale Funnel 提供	✓ 良好	公网流量加密

10.3 可靠性增强建议

项目	建议	操作
看门狗心跳	添加定时健康检查脚本，检测端口 18789 存活	创建 15 分钟间隔的计划任务，Test-NetConnection -Port 18789
日志监控	监控 OpenClaw 崩溃日志	gateway.cmd 输出重定向到日志文件
自动更新	npm update -g openclaw	手动或计划任务，建议先测试
备份配置	定期备份 openclaw.json	已通过 Git 仓库管理

11. 灾难恢复与重装指南

11.1 重装 Windows 后恢复 OpenClaw

```
# 1. 安装 Node.js LTS
winget install OpenJS.NodeJS.LTS

# 2. 安装 OpenClaw
npm install -g openclaw

# 3. 运行配置向导
openclaw onboard

# 4. 还原配置文件（从 Git 仓库或备份）
# openclaw.json 内含所有渠道/技能/模型配置

# 5. 设置环境变量
[System.Environment]::SetEnvironmentVariable('OPENCLAW_GATEWAY_PASSWORD', 'wlySecureClaw2026!', 'Machine')

# 6. 导入计划任务（管理员权限）
schtasks /Create /tn "OpenClaw Gateway" /xml "E:\RamdiskGuardian\openclaw_task.xml" /f

# 7. 验证
schtasks /run /tn "OpenClaw Gateway"
Start-Sleep -Seconds 10
Test-NetConnection -ComputerName 127.0.0.1 -Port 18789
```

11.2 换电脑迁移

1. 备份 `C:\Users\10979\openclaw\` 整个目录
2. 备份 `E:\RamdiskGuardian\` 中的所有脚本和 XML
3. 在新机器上按 11.1 步骤恢复
4. Tailscale 需要重新登录: `tailscale up --operator=$env:USERNAME`
5. Tailscale Funnel 需要重新启用: 在 Tailscale Admin Console 中授权 Funnel

12. 运维速查命令

```
# —— 状态检查 ——
# 查看网关进程
Get-Process node | Where-Object { $_.MainWindowTitle -eq '' } | Format-Table Id, CPU, WS

# 检查端口监听
netstat -an | Select-String ":18789"

# 查看计划任务状态
schtasks /query /tn "OpenClaw Gateway" /v /fo LIST

# Tailscale 状态
tailscale status
tailscale funnel status

# —— 手动操作 ——
# 手动启动网关
schtasks /run /tn "OpenClaw Gateway"

# 手动停止网关
Get-Process node | Where-Object { $_.CommandLine -match 'openclaw' } | Stop-Process

# 重启网关
Get-Process node | Where-Object { $_.CommandLine -match 'openclaw' } | Stop-Process; Start-Sleep 3; schtasks

# —— 环境变量 ——
# 查看密码 (Machine 级别)
[System.Environment]::GetEnvironmentVariable('OPENCLAW_GATEWAY_PASSWORD', 'Machine')

# 修改密码
[System.Environment]::SetEnvironmentVariable('OPENCLAW_GATEWAY_PASSWORD', '<新密码>', 'Machine')

# —— 日志与调试 ——
# 前台运行网关 (调试用)
& "C:\Users\10979\openclaw\gateway.cmd"

# 查看自愈脚本日志
Get-Content E:\RamdiskGuardian\logs\openclaw_guardian.log -Tail 30
```

13. 文件清单

本仓库中的 OpenClaw 相关文件

E:\RamdiskGuardian\	
├─ OPENCLAW.md	← 本文件（完整运维手册）
├─ OPENCLAW.pdf	← 本文件的 PDF 版本
├─ OPENCLAW_AUDIT.md	← 深度审计报告（审计发现+修复）
├─ openclaw_silent_boot_guardian.ps1	← 自愈脚本（审计+修复计划任务）
├─ openclaw_run_hidden.vbs	← VBScript 隐藏窗口启动器
├─ openclaw_task.xml	← 计划任务 XML 定义（可导入）

系统中的 OpenClaw 文件

C:\Users\10979\.openclaw\	
├─ openclaw.json	← 核心配置（渠道/模型/插件）
├─ gateway.cmd	← 启动脚本（OpenClaw 自动生成）
├─ workspace\	← 工作空间
│ └─ skills\	← 技能目录
│ └─ gemini\	
│ └─ github\	
│ └─ himalaya\	
│ └─ nano-pdf\	
│ └─ openai-whisper\	
│ └─ agent-browser-clawdbot\	
│ └─ self-improving\	
│ └─ session-logs\	
└─ memory\	← 记忆文件

14. 变更历史

日期	操作	说明
2026-06-16	● 计划任务全面重建	LogonTrigger → BootTrigger , InteractiveToken → S4U , 添加 HighestAvailable / Hidden / RestartOnFailure
2026-06-16	● 环境变量提升	OPENCLAW_GATEWAY_PASSWORD 从 User 级别提升至 Machine 级别
2026-06-16	● 文档创建	编写完整运维手册、审计报告
2026-06-15	初始安装	OpenClaw v2026.6.6 首次配置, openclaw onboard 向导完成